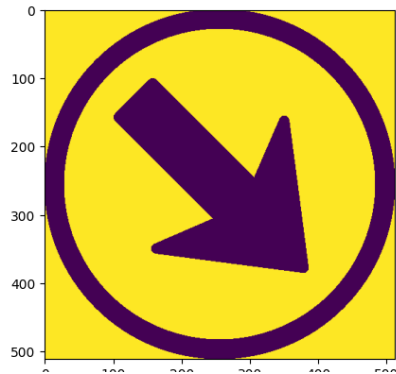


```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files
upload_image = files.upload()
image = cv2.imread('32215.png')
img = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
plt.imshow(img)
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving 32215.png to 32215.png
<matplotlib.image.AxesImage at 0x7d06b01f5cd0>

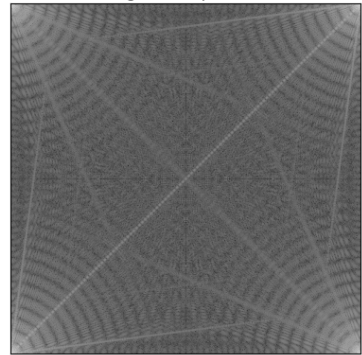


Step 1: Fast Fourier Transform (FFT)

```
f = np.fft.fft2(img) #the image 'img' is passed to np.fft.fft2() to compute its 2D Discrete Fourier transform f
mag = 20*np.log(np.abs(f))
plt.imshow(mag, cmap = 'gray') #cmap='gray' parameter to indicate that the image should be displayed in grayscale.
plt.title('Magnitude Spectrum')
plt.xticks([], plt.yticks([]))
plt.show()
```

/tmp/ipykernel_16262/3724294990.py:2: RuntimeWarning: divide by zero encountered in log
mag = 20*np.log(np.abs(f))

Magnitude Spectrum



The magnitude of the Fourier transform f is computed using $\text{np.abs}()$, converted to a logarithmic scale using $\text{np.log}()$ and multiplied by 20 to get the magnitude in decibels. This is done to make the magnitude spectrum easier to visualize and interpret

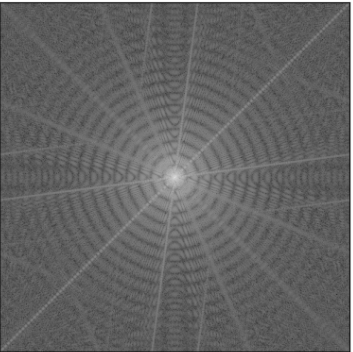
Step 2: FFT Shift

The zero-frequency component of the image is shifted to the center of the frequency spectrum utilizing an FFT shift.

```
fshift = np.fft.fftshift(f)
mag = 20*np.log(np.abs(fshift))
plt.imshow(mag, cmap = 'gray')
plt.title('Centered Spectrum'), plt.xticks([], plt.yticks([]))
plt.show()
```

/tmp/ipykernel_16262/584503348.py:2: RuntimeWarning: divide by zero encountered in log
mag = 20*np.log(np.abs(fshift))

Centered Spectrum



Step 3: Filtering

Functions to Generate an Image of a Disk for Ideal High and Low pass filters with Desired Number of Pixels

```
import math
def distance(point1,point2):
    return math.sqrt((point1[0]-point2[0])**2 + (point1[1]-point2[1])**2)

def idealFilterLP(D0,imgShape):
    base = np.zeros(imgShape[:2])
    rows, cols = imgShape[:2]
    center = (rows/2,cols/2)
    for x in range(cols):
        for y in range(rows):
            if distance((y,x),center) < D0:
                base[y,x] = 1
    return base

def idealFilterHP(D0,imgShape):
    base = np.ones(imgShape[:2])
    rows, cols = imgShape[:2]
    center = (rows/2,cols/2)
    for x in range(cols):
        for y in range(rows):
            if distance((y,x),center) < D0:
                base[y,x] = 0
    return base
```

```
import math
def distance(point1,point2):
    return math.sqrt((point1[0]-point2[0])**2 + (point1[1]-point2[1])**2)

def gaussianLP(D0,imgShape):
    base = np.zeros(imgShape[:2])
    rows, cols = imgShape[:2]
    center = (rows/2,cols/2)
    for x in range(cols):
        for y in range(rows):
            base[y,x] = math.exp((-distance((y,x),center)**2)/(2*(D0**2)))
    return base

def gaussianHP(D0,imgShape):
    base = np.zeros(imgShape[:2])
    rows, cols = imgShape[:2]
    center = (rows/2,cols/2)
```

```
for x in range(cols):
    for y in range(rows):
        base[y,x] = 1 - math.exp(((distance((y,x),center)**2)/(2*(D0**2))))
    return base
```

Sample Filters

```
fig, ax = plt.subplots(2, 2) # create a 2x2 grid of subplots
fig.suptitle('Filters') # set the title for the entire figure

# plot the first image in the top-left subplot
im1 = ax[0, 0].imshow(np.abs(idealFilterLP(50, img.shape)), cmap='gray')
ax[0, 0].set_title('Low Pass Filter of Diameter 50 px')
ax[0, 0].set_xticks([])
ax[0, 0].set_yticks([])

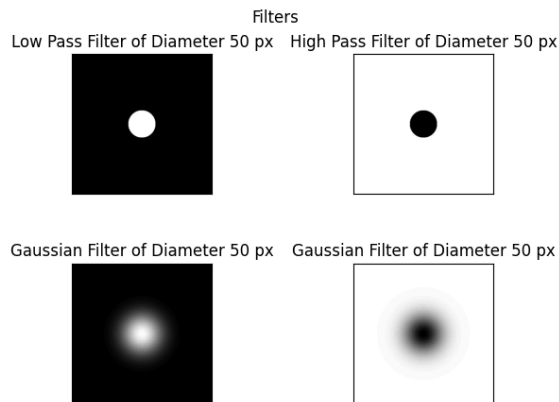
# plot the second image in the top-right subplot
im2 = ax[0, 1].imshow(np.abs(idealFilterHP(50, img.shape)), cmap='gray')
ax[0, 1].set_title('High Pass Filter of Diameter 50 px')
ax[0, 1].set_xticks([])
ax[0, 1].set_yticks([])

# plot the third image in the bottom-left subplot
im3 = ax[1, 0].imshow(np.abs(gaussianLP(50, img.shape)), cmap='gray')
ax[1, 0].set_title('Gaussian Filter of Diameter 50 px')
ax[1, 0].set_xticks([])
ax[1, 0].set_yticks([])

# plot the fourth image in the bottom-right subplot
im4 = ax[1, 1].imshow(np.abs(gaussianHP(50, img.shape)), cmap='gray')
ax[1, 1].set_title('Gaussian Filter of Diameter 50 px')
ax[1, 1].set_xticks([])
ax[1, 1].set_yticks([])

# adjust the spacing between subplots
fig.subplots_adjust(wspace=0.5, hspace=0.5)

# save the figure to a file
fig.savefig('filters.png', bbox_inches='tight')
```

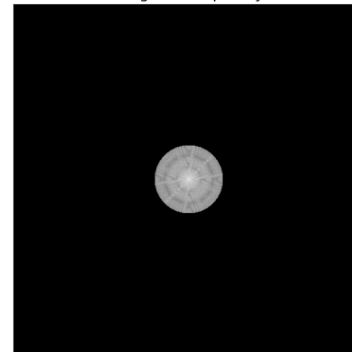


Step 4: Multiplying Filter and Shifted Image to Get Filtered Image

```
fig, ax = plt.subplots()
im = ax.imshow(np.log(1+np.abs(fshift * idealFilterLP(50,img.shape)))), cmap='gray')
ax.set_title('Filtered Image in Frequency Domain')
ax.set_xticks([])
ax.set_yticks([])

fig.savefig('filtered image in freq domain.png', bbox_inches='tight')
```

Filtered Image in Frequency Domain



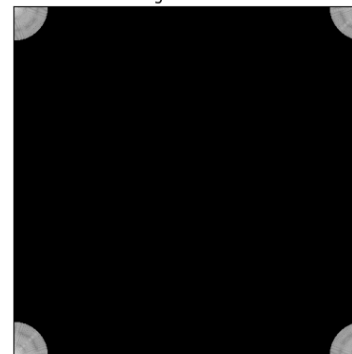
Step 5: Inverse FFT Shift

After filtering the image in the frequency domain, we need to move it back to its original location before applying the inverse FFT. To achieve this, we use the inverse FFT shift, which reverses the shifting process performed earlier.

```
fig, ax = plt.subplots()
im = ax.imshow(np.log(1+np.abs(np.fft.ifftshift(fshift * idealFilterLP(50,img.shape)))), cmap='gray')
ax.set_title('Filtered Image inverse fft shifted')
ax.set_xticks([])
ax.set_yticks([])
```

```
fig.savefig('filtered image inverse fft shifted.png', bbox_inches='tight')
```

Filtered Image inverse fft shifted



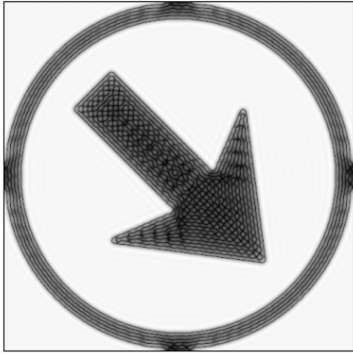
Step 6: Inverse Fast Fourier Transform (IFFT)

The Inverse Fast Fourier Transform (IFFT) is the final step in the frequency transformation of images. It is used to transfer the image from the frequency domain back to the spatial domain. This step results in an image that has reduced noise and improved clarity compared to the original image in the spatial domain.

```
fig, ax = plt.subplots()
im = ax.imshow(np.log(1+np.abs(np.fft.ifft2(np.fft.ifftshift(fshift * idealFilterLP(50,img.shape))))), cmap='gray')
ax.set_title('Final Filtered Image In Spatial Domain')
ax.set_xticks([])
ax.set_yticks([])
```

```
fig.savefig('final filtered image.png', bbox_inches='tight')
```

Final Filtered Image In Spatial Domain



Function to Plot all the images

```
def Freq_Trans(image, filter_used):
    img_in_freq_domain = np.fft.fft2(image)

    # Shift the zero-frequency component to the center of the frequency spectrum
    centered = np.fft.fftshift(img_in_freq_domain)

    # Multiply the filter with the centered spectrum
    filtered_image_in_freq_domain = centered * filter_used

    # Shift the zero-frequency component back to the top-left corner of the frequency spectrum
    inverse_fftshift_on_filtered_image = np.fft.ifftshift(filtered_image_in_freq_domain)

    # Apply the inverse Fourier transform to obtain the final filtered image
    final_filtered_image = np.fft.ifft2(inverse_fftshift_on_filtered_image)

    return img_in_freq_domain, centered, filter_used, filtered_image_in_freq_domain, inverse_fftshift_on_filtered_image, final_filtered_image
```

For this project, selected diameters are 50, 100, and 150 pixels for both high-pass and low-pass Ideal and Gaussian filters individually, in order to showcase their impact on enhancing image sharpness.

```
fig, axs = plt.subplots(12, 7, figsize=(30, 60))

filters = [(f, d) for f in [idealFilterLP, idealFilterHP, gaussianLP, gaussianHP] for d in [50, 100, 150]]

for row, (filter_name, filter_diameter) in enumerate(filters):
    # Plot each filter output on a separate subplot
    result = Freq_Trans(img, filter_name(filter_diameter, img.shape))

    for col, title, img_array in zip(range(7),
    ["Original Image", "Spectrum", "Centered Spectrum", f"{filter_name.__name__} of Diameter {filter_diameter} px",
    f"Centered Spectrum multiplied by {filter_name.__name__}", "Decentralize", "Processed Image"],
    [img, np.log(1+np.abs(result[0])), np.log(1+np.abs(result[1])), np.abs(result[2]), np.log(1+np.abs(result[3])),
    np.log(1+np.abs(result[4])), np.abs(result[5]))]):

        axs[row, col].imshow(img_array, cmap='gray')
        axs[row, col].set_title(title)

plt.tight_layout()
plt.savefig('all_processess.png', bbox_inches='tight')
plt.show()
```

